

Fantasy Draft Assistant — Requirements

Owner: Blair **Target:** Local Python + Streamlit app, run on draft day (~mid-September 2026)

Purpose: During a live 10-team PPR snake draft, recommend the top 3 picks that maximize the projected points of my final starting lineup, accounting for analyst disagreement and for what my opponents are likely to do before my next turn.

1. League configuration

These are hard values. Put them in `config.py` so they can be changed, but these are the defaults.

Setting	Value
Teams	10
Rounds	16 (160 total picks)
Draft type	Snake
My draft slot	Unknown until draft day — must be a setting, selectable 1–10 at launch
Keepers	None
Roster size	16 (9 starters + 7 bench)
Starting lineup	1 QB, 2 RB, 2 WR, 1 TE, 1 FLEX (RB/WR/TE), 1 D/ST, 1 K
Scoring	ESPN standard PPR (full detail in §4)

Snake pick math

For slot `s` (1-indexed) in a 10-team league, round `r`:

odd round: $\text{pick} = (r - 1) * 10 + s$

even round: $\text{pick} = (r - 1) * 10 + (11 - s)$

Example, slot 3: picks 3, 18, 23, 38, 43, 58, 63, 78, 83, 98, 103, 118, 123, 138, 143, 158.

The gap between consecutive picks alternates (15 picks, then 5 picks, at slot 3). **The recommendation engine must use the real gap, not an average** — the whole point of the app is knowing who survives to the next turn.

2. Scope

In scope (v1)

- Load analyst projections + rankings from local snapshot files
- Compute consensus value and analyst disagreement per player
- Manual click-off draft board tracking all 160 picks
- Monte Carlo simulation of the remaining draft
- Top-3 pick recommendation with plain-English reasoning
- 3 seasons of historical scoring per player with game-level drill-down
- Draft state auto-save / crash recovery

Out of scope (v1) — do not build these

- Live ESPN API sync (manual board only)
 - Bye-week optimization
 - Injury status / news feeds
 - Handcuff or stack logic
 - Auction drafts, dynasty, keeper leagues
 - Multi-user / hosted deployment
-

3. Data sources

Acquisition model: snapshot files. Everything is downloaded/exported once before draft day into **data/raw/** and committed. The app reads local files at launch. **No network calls are required for the app to run.** This is a hard requirement — draft day cannot depend on a scraper working.

3.1 Analyst projections & rankings

Panel = the top-accuracy expert ranking sources from the 2025 season per FantasyPros' accuracy scoring. Target **at least 6** so the standard deviation is meaningful:

#	Expert	Site
1	Seth Miller	Crossroads Fantasy Football
2	Guilherme Gianni	Fantasy Futebolista
3	Michael Bobal	SquareEdgeMike / The 33rd Team
4	Jason Willan	Gridiron Experts
5	Marc Shannep	Fantasy Knockout
6	Joey Wright	Footballguys
7	Ryan Weisse	Club Fantasy FFL
8	Kevin Steele	The Fantasy Authority
9	Jody Smith	Draft Sharks
10	Tim Heaney	Sporting News

Fallback if per-expert rankings aren't individually exportable: FantasyPros' consensus rankings pages already publish **Best**, **Worst**, **Avg**, and **Std Dev** columns per player across their full expert panel. Those four columns satisfy the entire disagreement requirement with one file. Use per-expert files if obtainable; otherwise use the consensus file's spread columns and note the substitution in the UI.

Also needed: a points-projection source (ranks alone can't produce a lineup total). FantasyPros projections pages and FFToday both publish projected stat lines. At minimum we need projected PPR points per player.

3.2 Opponent behavior model

ESPN's own player rankings / ADP — correct source because the league auto-drafts off ESPN's board. Export to [data/raw/espn_adp.csv](#).

3.3 Historical scoring

Use **nflreadpy** (the maintained Python port of nflreadr; **nfl_data_py** is the older package).

```
import nflreadpy as nfl
```

```
weekly = nfl.load_player_stats(seasons=[2023, 2024, 2025]) # returns Polars
```

```
ids = nfl.load_ff_playerids() # cross-platform ID map
```

Cache both to `data/cache/*.parquet` on first run so the app never needs the network again.

Important: compute PPR points ourselves from the raw stat lines using §4 rules. Do not use any source's precomputed fantasy points — they won't match our exact scoring.

4. Scoring engine

Implement as a pure function: `stat_line -> points`. This is used for historical points AND for converting projected stat lines into projected points, so both are consistent.

Passing: $0.04 \text{ pts/yd} \cdot \text{TD} \cdot 4 \cdot \text{INT} \cdot -2 \cdot 2\text{PC} \cdot 2$ **Rushing/Receiving:** $\text{reception} \cdot 1.0 \cdot 0.1 \text{ pts/yd} \cdot \text{TD} \cdot 6 \cdot 2\text{PC} \cdot 2 \cdot \text{fumble lost} \cdot -2$ **Kicking:** $\text{PAT} \cdot 1 \cdot \text{FG } 0\text{--}39 = 3 \cdot \text{FG } 40\text{--}49 = 4 \cdot \text{FG } 50+ = 5 \cdot \text{misses} \cdot 0$ **D/ST:** $\text{sack} \cdot 1 \cdot \text{INT} \cdot 2 \cdot \text{fumble rec} \cdot 2 \cdot \text{safety} \cdot 2 \cdot \text{blocked kick} \cdot 2 \cdot \text{TD} \cdot 6$ **D/ST points allowed:** $0 \rightarrow 5 \cdot 1\text{--}6 \rightarrow 4 \cdot 7\text{--}13 \rightarrow 3 \cdot 14\text{--}17 \rightarrow 1 \cdot 18\text{--}27 \rightarrow 0 \cdot 28\text{--}34 \rightarrow -1 \cdot 35+ \rightarrow -3$

Unit-test this function against 5–10 hand-calculated stat lines before anything else is built.

5. Player identity resolution

The highest-risk plumbing in the project. "Marvin Harrison Jr." vs "Marvin Harrison Jr" vs "M. Harrison" will silently corrupt the variance math by splitting one player into two rows.

Requirements:

- Use `nfl.load_ff_playerids()` as the canonical crosswalk (it carries ESPN, Sleeper, FantasyPros, and other platform IDs)
- Normalize: lowercase, strip punctuation and suffixes (Jr/Sr/II/III), collapse whitespace
- Match on `(normalized_name, position, team)`, fall back to fuzzy match with a similarity threshold
- **Every unmatched player must be reported, not silently dropped.** Write `data/unmatched.csv` and show a warning banner in the UI with the count
- Support `data/manual_id_overrides.csv` (`source_name`, `canonical_id`) for hand-fixing stragglers
- Acceptance: after loading all sources, unmatched players in the top 200 by consensus rank must be **zero**

6. Core player metrics

Per player, compute:

Field	Definition
<code>proj_points</code>	Consensus projected PPR points for the season
<code>consensus_rank</code>	Average rank across the analyst panel
<code>rank_sd</code>	Standard deviation of ranks across the panel — the disagreement measure
<code>rank_best / rank_worst</code>	Range across the panel
<code>points_sd</code>	Projected-points standard deviation. If only ranks are available, derive it by mapping the rank range onto the points curve at that position
<code>espn_adp</code>	ESPN rank/ADP, used only for the opponent model
<code>hist_3yr</code>	Total PPR points and points-per-game for 2023, 2024, 2025

Your example — a player at 1,3,2,2,3 vs one at 1,8,2,2,3 — is exactly `rank_sd`. Both have similar averages; the second has a much wider spread and should be treated as riskier.

7. Opponent model (Monte Carlo)

Each simulated draft:

1. Perturb the board once per simulation: `adp_sim[i] = espn_adp[i] + N(0, sigma)`, `sigma` $\approx$ 8 picks. Perturbing once per sim (not once per pick) models "this year's board leans a particular way," which is more realistic than independent per-pick noise.

2. Each opponent pick takes the lowest `adp_sim` player still available, subject to roster sanity caps: max 2 QB, max 2 TE, max 1 K, max 1 D/ST, max 16 total; no K or D/ST before round 13.
3. Opponents do not model need beyond those caps. Keep it simple — the caps prevent the obviously-wrong outcomes and nothing else is worth the complexity.

Sigma must be a tunable setting. A `sigma = 0` mode (strict ESPN order) is required as a debugging and comparison view.

8. Recommendation engine — full roster simulation

This is the heart of the app. It answers the literal question: *which available player leads to the best final starting lineup?*

```
def recommend(board, my_roster, my_remaining_picks, n_sims=500, k=10, risk_pct=50):
```

```
    candidates = top_k_available(board, k)      # by consensus value, positionally diverse
```

```
    results = {}
```

```
    for c in candidates:
```

```
        outcomes = []
```

```
        for _ in range(n_sims):
```

```
            proj = draw_projections(board)      # N(proj_points, points_sd) per player
```

```
            adp_s = perturb_adp(board)         # §7 step 1
```

```
            avail = board - {c}
```

```
            roster = my_roster + [c]
```

```
            for pick_no in remaining_picks_in_order:
```

```
                if pick_no in my_remaining_picks:
```

```
                    p = greedy_my_pick(avail, roster, proj) # §8.1
```

```
                else:
```

```

        p = opponent_pick(avail, adp_s)      # §7 step 2

        avail.remove(p)

        if mine: roster.append(p)

        outcomes.append(optimal_starting_lineup_points(roster, proj)) # §8.2

    results[c] = percentile(outcomes, risk_pct)

    return sorted(results, reverse=True)[:3]

```

The TE-cliff scenario you described falls out of this automatically. No hand-coded scarcity rules — if taking a TE now produces better final rosters across 500 simulated futures, it ranks first.

8.1 Future-me policy

Simulated future-me picks greedily: highest projected points among players that can still fill an unfilled starting slot; once starters are filled, best available at RB/WR/TE. Deliberately simple and self-consistent — do **not** make this recursive.

8.2 Lineup optimizer

Given 16 players and their simulated point totals, fill 1 QB / 2 RB / 2 WR / 1 TE / 1 FLEX / 1 D/ST / 1 K to maximize total. Greedy by position then best remaining RB/WR/TE into FLEX is provably optimal for this slot structure — no solver needed.

8.3 Risk dial

The variance slider operates on the **team outcome distribution**, not on individual players:

- **Safe (30th percentile):** optimize the floor
- **Balanced (50th):** optimize the median — default
- **Upside (70th):** optimize the ceiling

Rationale: a boom/bust WR4 barely moves your team; a boom/bust RB1 moves it a lot. Applying risk at the team level captures that; a per-player penalty does not.

8.4 Also display

- **Availability odds:** for each candidate and each position, $P(\text{a player of this tier survives to my next pick})$, straight from the simulations
- **Separation:** my projected starting lineup total vs the average simulated opponent's

9. Performance requirements

- **Recommendation must return in under 3 seconds.** Non-negotiable — there is a draft clock.
 - Represent the player pool as **NumPy arrays indexed by integer player ID**. No pandas or Polars operations inside the simulation loop. This is the single most common way this kind of code ends up 100× too slow.
 - Precompute everything possible at load: position masks, sorted ADP order, projection arrays
 - Vectorize across simulations where possible (run all `n_sims` for a candidate as array operations rather than a Python loop)
 - If it can't hit 3 seconds, reduce `n_sims` to 250 and expose it as a setting rather than accepting slow
-

10. UI (Streamlit)

Single main screen, optimized for reading fast under a clock.

Top: recommendation cards (3, side by side)

Each shows: player name, position, team · recommendation score · availability odds at my next pick · **one sentence of plain-English reasoning**, e.g.

"TE cliff — 88% chance no top-3 TE survives to pick 38, and the gap to TE7 is 41 points."

Middle: draft board

- Search box + position filters
- Click a player to mark drafted; a dropdown assigns which team took him (defaults to whoever is on the clock)
- Current pick number, round, whose turn, and picks until my next turn always visible
- **Undo last pick** button — mis-clicks will happen
- My roster panel showing filled and unfilled starting slots

Bottom / expandable: player detail

- 2023 / 2024 / 2025 total PPR points and points per game
- Game-by-game log for any selected season, as a table and a bar chart

- Consistency: % of games ≥ 20 pts (boom) and ≤ 5 pts (bust), games missed
- Analyst detail: each analyst's rank, plus best/worst/avg/std dev

Sidebar: settings

Draft slot (1–10) · risk dial (Safe / Balanced / Upside) · sigma · n_sims · engine toggle
(Simulation vs VONA, see §11)

Persistence

Write full draft state to `state/draft_YYYYMMDD.json` after **every** pick. On launch, offer to resume if a state file exists. If the app crashes at pick 90, nothing is lost.

11. Build phases

Build in this order. Each phase must be working and committed before the next starts.

Phase 0 — Foundation Scoring function + unit tests · data loaders · player ID resolution · `data/unmatched.csv` clean for top 200 *Accept*: scoring tests pass; zero unmatched in top 200

Phase 1 — Static board Streamlit UI, player table with consensus value and spread, click-off tracking, undo, state save/resume, my-roster panel *Accept*: can manually run a full 160-pick mock draft without the app breaking

Phase 2 — VONA engine (the safety net) Simple recommendation: best available now vs best likely available at my next pick, per position *Accept*: returns sane top 3 instantly; **this is the guaranteed-working draft-day fallback**

Phase 3 — Simulation engine Monte Carlo opponent model, full roster simulation, risk dial, availability odds, reasoning strings *Accept*: meets the 3-second budget; toggle lets me compare against VONA

Phase 4 — History drill-down 3-year totals, game logs, consistency metrics, charts

Phase 5 — Polish Reasoning wording, layout under time pressure, dry-run mock draft

If time runs short, **ship through Phase 2 and stop**. A working simple app beats a half-finished clever one on draft day.

12. Testing & verification

- **Scoring unit tests** — hand-calculated stat lines, including D/ST tiers and FG distance bands
 - **Snake pick math test** — assert slot 3 yields picks [3, 18, 23, 38, ...] and slot 10 yields [10, 11, 30, 31, ...]
 - **Lineup optimizer test** — hand-built roster with a known optimal lineup
 - **The TE-cliff scenario test** — construct a synthetic board with 2 elite TEs, a steep TE dropoff, and 12 near-identical RBs. Assert the engine recommends a TE. *This is the behavior the whole app exists for; it deserves a test.*
 - **Determinism** — fixed random seed produces identical recommendations, so results are debuggable
 - **Sanity check** — with `sigma = 0`, opponent picks must exactly reproduce ESPN order
 - **Cross-check** — run Phase 2 and Phase 3 engines side by side on a mock draft. Large disagreements mean a bug in one of them; investigate before draft day
 - **Full dry run** — complete a mock draft end to end at least 3 days before the real one
-

13. Known risks

Risk	Mitigation
ADP model is wrong → confidently wrong recommendations	Sigma is tunable; show availability odds so I can apply judgment
Simulation too slow under the clock	Phase 2 VONA fallback always available; <code>n_sims</code> adjustable
Name matching silently splits players	Unmatched report + manual override file + zero-unmatched acceptance test
Projections themselves are noisy	Risk dial; disagreement shown, not hidden
Mis-click during live draft	Undo button + per-pick state save
Source site changes format before draft day	Snapshot files are frozen locally; no live dependency

14. Open items

1. Confirm whether per-expert rankings for the 10 named analysts can actually be exported, or whether we fall back to FantasyPros' consensus Best/Worst/Avg/StdDev columns
2. Confirm the points-projection source (needed for lineup totals, not just ranks)
3. Decide `sigma` default by back-testing against a real 2025 draft board if one is available
4. Kicker and D/ST projections are close to noise — consider treating them as fixed late-round filler rather than simulating them

Sources

- [2025's Most Accurate Fantasy Football Draft Rankings — FantasyPros](#)
- [FantasyPros Draft Accuracy Scores](#)
- [FantasyPros PPR Cheat Sheet](#)
- [ESPN Fantasy Projections](#)
- [FFToday Rankings](#)
- [nflreadpy load functions](#)
- [nflverse/nflreadpy on GitHub](#)